

Inventory Planning Agent

Deploying an AI-powered reorder assistant on Azure

Architecture · Deployment · Cost · Operations

LangChain · Groq / Llama 3.3 · Azure Container Apps · Gradio

What We Built

A chat interface where users describe their inventory and receive AI-generated reorder decisions — backed by live data.



Chat Interface

Gradio web UI
Public URL
Users type plain-English questions



AI Agent

LangChain + Llama 3.3
Runs on Groq
Two tools: DB lookup + reorder calc



REST API

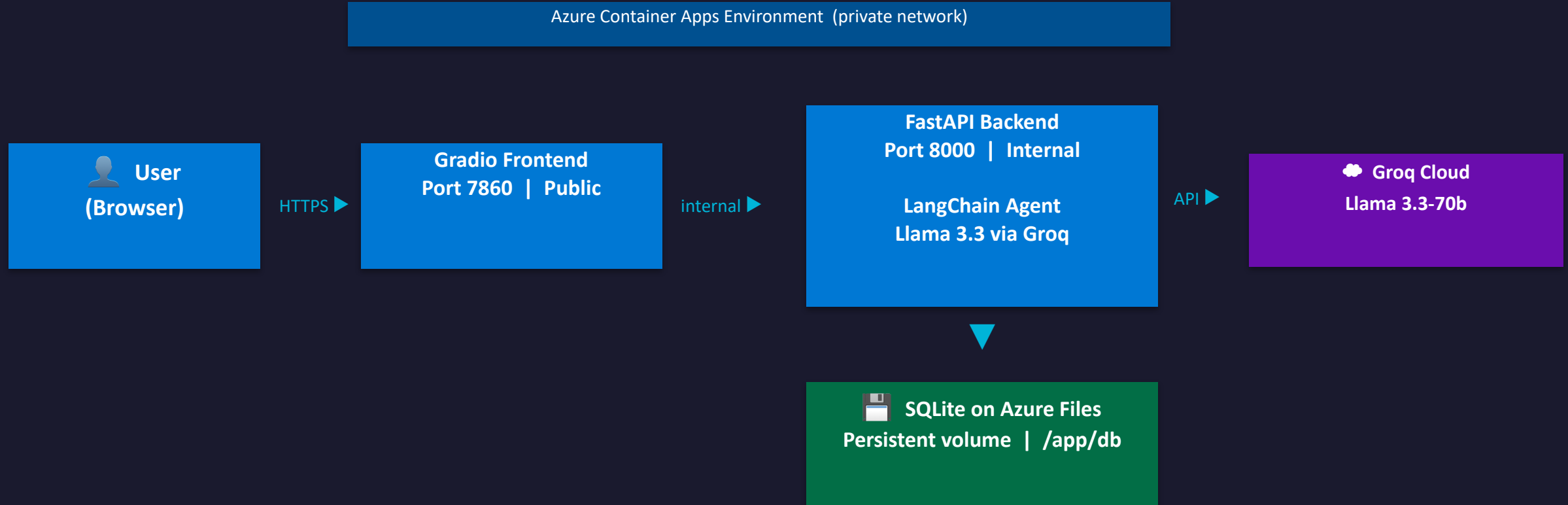
FastAPI backend
Internal-only (not public)
API key protected



Persistent DB

SQLite on Azure Files
Inventory + session history
Survives restarts & redeployments

Architecture



Key design decisions:

- ▶ Frontend is the only public entry point — backend never exposed to internet
- ▶ Frontend and backend share a private network — internal calls are faster and free
- ▶ Database lives on Azure Files — persists independently of container lifecycle

Deployment Strategy — Why We Version Everything

Each deployment gets a fresh resource group (v1, v2, v3...). Only the Container Registry is shared.

✓ What changes each version

Resource group → inventory-rg-v2

Storage account → inventorysav2

Container Apps environment

Image tags → :v2

Backend + frontend Container Apps

🔒 What stays the same

Azure Container Registry (inventoryacr2)

Subscription (demoproj)

API keys and secrets

Application code (only image tag changes)

💡 *v1 keeps running while v2 is deployed and tested — instant rollback if needed. Delete v1 only once v2 is confirmed.*

11-Step Deployment at a Glance

1

Login

az login + select subscription

2

Resource Group

Fresh container for all v-N resources

3

Build & Push Images

Docker build → push to ACR (inventoryacr2)

4

Storage + File Share

Azure Files for persistent SQLite DB

5

Container Env

Shared private network for both containers

6

Link File Share

Register storage credentials with the environment

7

Deploy Backend

Create app (CLI flags) + patch volume (REST API)

8

Get Backend URL

Internal URL — not reachable from internet

9

Validate

Test from Azure Cloud Shell only

10

Deploy Frontend

Public Gradio UI — passes backend URL as env var

11

Get Public URL

Share with users — done

The Persistent Database — Why It Needs Special Handling

✗ Without a persistent volume

Container storage is ephemeral by default
Every restart wipes the database
Redeployment = all inventory data lost
Session history gone

✓ With Azure Files volume mount

Azure Files share mounted at /app/db
Database file lives outside the container
Survives restarts, redeployments, crashes
inventory.db persists across v1 → v2 → v3

How it works:

- ▶ Step 4: Create Azure Storage Account + File Share (inventorydb)
- ▶ Step 6: Register storage credentials with the Container Apps Environment
- ▶ Step 7b: Patch the backend container to mount the share at /app/db via REST API
- ▶ On startup: init_db.py runs — creates schema and seeds sample data if not present (idempotent)

⚠ *Note: az containerapp create --yaml has a bug in the current CLI extension — volume mount is applied via az rest PATCH instead.*

Security Model



Backend never public

Deployed with --ingress internal. Only containers in the same environment can reach it. No public IP.



API key authentication

Every request from the frontend must include X-API-Key header. Requests without it are rejected with 403.



Secrets never in plain text

GROQ_API_KEY and API_KEY are stored as Container Apps secrets. They appear as secretRef in config, not as plain env vars.



Registry is private

Images are stored in inventoryacr2 (Azure Container Registry) with admin auth. Not on Docker Hub.



Frontend is the only entry point

Users interact only with the Gradio UI. The AI agent, database, and API are all shielded behind it.

Cost — 3 Days, ~20 Users, ~30 Queries

Always-on (min-replicas 1)	
Backend (1 vCPU, 2Gi)	~\$7.80
Frontend (0.5 vCPU, 1Gi)	~\$3.90
Container Registry (Basic)	~\$0.50
Azure Storage	~\$0.01
Groq API (30 queries)	\$0.00
Total	~\$12.25

Scale-to-zero (min-replicas 0)	
Backend (only while processing)	~\$0.02
Frontend (only while active)	~\$0.01
Container Registry (Basic)	~\$0.50
Azure Storage	~\$0.01
Groq API (30 queries)	\$0.00
Total	~\$0.54

💡 *Scale-to-zero recommended for demos and low-traffic use. Trade-off: 10-20 second cold start after idle period.*

Operations & Troubleshooting

View live logs

```
az containerapp logs show -n inventory-frontend -g inventory-rg-v3 --follow
```

Check system events

```
az containerapp logs show -n inventory-backend -g inventory-rg-v3 --type system --tail 50
```

Update frontend only

```
az containerapp update -n inventory-frontend -g inventory-rg-v3 --image inventoryacr2.azurecr.io/inventory-frontend:v4
```

Scale to zero (cost saving)

```
az containerapp update -n inventory-backend -g inventory-rg-v3 --min-replicas 0
```

Delete full deployment

```
az group delete --name inventory-rg-v3 --yes --no-wait
```

⚠ *Backend is internal-only — validate via Azure Cloud Shell, not your local terminal.*

What's Next — Scaling the Architecture



Swap SQLite → Azure SQL / Cosmos DB

Change DB_PATH env var and db_client.py connection. No other changes needed. Supports multiple concurrent users and larger datasets.



Add user authentication

Add Azure Active Directory (Entra ID) in front of the Gradio frontend. Users log in with their org credentials.



Multi-agent workflows

Add specialist agents (demand forecasting, supplier selection). LangChain's AgentExecutor supports tool chaining across agents.



Analytics & monitoring

Sessions table already logs every query and response. Connect Power BI or Azure Monitor to the Log Analytics workspace created automatically.



CI/CD pipeline

GitHub Actions: on push → build image → push to ACR → az containerapp update. Full automation of Steps 3 + update.